

TimeLens AI: A Personalized Time-Prediction and Adaptive Scheduling Engine

Software Engineering Project Proposal

Vansh Jain (1024030770) Devyang (1024030765) Bhavik (1024030772)

Department of Computer Engineering
Thapar Institute of Engineering and Technology

August 24, 2026

1 Higher-Order Goal

This project aims to enhance personal productivity and temporal literacy by replacing conventional, static task planning with an adaptive, data-driven forecasting architecture. Beyond immediate technical execution, it targets a fundamental systemic issue in productivity software: human time misestimation follows personal, measurable, and systemic patterns. Existing task managers treat user-inputted durations as absolute ground truth, completely ignoring historical behavioral signals.

2 Problem Statement

Current task management platforms function primarily as passive storage systems, relying on user-provided completion estimates without validation. This static operational model fails due to five critical structural flaws:

- **Unmitigated Estimation Bias:** Users systematically under- or overestimate durations. Planners fail to track or adjust for these individual, category-specific variance patterns.
- **Assumption of Uniform Productivity:** Human cognitive output fluctuates based on circadian rhythms, energy reserves, and continuous workload, whereas planners assume static throughput.
- **Uncaptured Context-Switching Costs:** External interruptions and task switching degrade operational efficiency, yet existing systems fail to factor these delays into future allocations.
- **Unmodeled Deadline Dynamics:** Dynamic responses to approaching deadlines, ranging from output acceleration to behavioral paralysis, are entirely unaccounted for.
- **Static Schedule Degradation:** Schedules remain rigid following initial delays, compounding downstream timeline failures rather than adaptively re-balancing workload.

TimeLens AI resolves these structural limitations by constructing per-user statistical models that estimate realistic task completion vectors and automatically re-sequence remaining workloads upon data ingestion.

3 Objectives

- Engine construction for personalized task duration estimation localized per user and task category.
- Algorithmic quantification and auto-correction of user-specific estimation bias profiles.
- Modeling time-of-day productivity efficiency and cumulative cognitive workload trends.
- Integration of quantifiable task interruption and context-switch overhead metrics.
- Implementation of a real-time reactive scheduling engine for dynamic workload re-allocation.

4 Methodology & Solution Approach

4.1 System Architecture & Pipeline

The end-to-end operational pipeline processes tasks through four core sequential phases:

1. **Ingestion & Feature Extraction:** Accepts inputs via Client API Gateway. Computes rolling user-category estimation biases, temporal productivity profiles, interruption metrics, and deadline proximity factors.
2. **Predictive Inference:** Evaluates extracted vectors using a machine learning engine (transitioning from baseline linear models to gradient-boosted regressors as data density scales) to generate predicted completion durations and confidence intervals.
3. **Dynamic Schedule Re-alignment:** Re-sequences and recalibrates open tasks systematically upon trigger events (completion, creation, or delay).
4. **Feedback Loop Persistence:** Commits actual completion metrics to an immutable historical repository, continuously refining parameters.

4.2 Key Model Features

Features analyzed include: Task Category, Self-Reported Estimate, Historical User-Category Estimation Bias, Start Timestamp (Circadian Vector), Daily Cumulative Workload, Recorded Interruption Volume, and Temporal Distance to Deadline.

4.3 Personalization & Cold-Start Strategy

Models prioritize individual operational history over global heuristics. For zero-history onboarding, the framework applies a population-level prior, continuously blending and shifting weights toward the personalized statistical model as empirical task data accumulates.

4.4 Technical Infrastructure

- **Core Services:** RESTful API Architecture with Token-based Authentication.
- **ML Engine:** `scikit-learn` / `XGBoost` gradient-boosted regressor models.
- **Data Tier:** Relational Store for tasks and history; Feature Store for rolling statistical profiles.
- **User Interface:** Real-time visualization dashboard tracking historical bias and live schedules.

5 Evaluation Criteria

Target performance is evaluated strictly through quantitative KPIs:

- **Prediction Accuracy:** Mean Absolute Error (MAE) between predicted vs. actual durations.

- **Bias Reduction:** Variance reduction between user estimates and empirical durations post-correction.
- **Schedule Adherence:** Percentage of tasks executed within the dynamically projected window.
- **Rebuild Latency:** Execution latency of schedule recalibration upon state change.
- **Missed-Deadline Reduction:** Metric shift in breached deadlines against baseline.

Execution Case Example: A user estimates 60 mins for a technical report scheduled for the afternoon. Incorporating historical bias and circadian throughput degradation, *TimeLens AI* predicts 95 mins, automatically delaying subsequent downstream tasks by 35 mins to maintain schedule integrity.

6 Scalability & System Architecture

- **Decoupled Infrastructure:** Ingestion, inference, and scheduling operate as isolated microservices to ensure horizontal scale.
- **Stateless Execution:** Database optimization through indexed user querying and per-user model training guarantees isolated computational overhead.

7 Resource Allocation

Implementation relies exclusively on open-source frameworks (`scikit-learn`, relational DBs) and existing university infrastructure, requiring zero external funding.

8 Project Scope & Deliverables

8.1 Phase 1 Deliverables

- Task management and authentication API endpoints.
- Baseline user-category estimation bias correction engine.
- Primary duration prediction endpoint and core telemetry dashboard.

8.2 Phase 2 Deliverables

- Integrated circadian productivity and interruption metrics engine.
- Dynamic schedule rebuilding system with deadline-pressure scaling.
- Automated persistent learning loop and live analytics reporting.

9 Risk Matrix & Mitigation Strategies

Risk Factor	Mitigation Strategy
Cold-Start Anomalies	Initialize with category-level population priors; decay weights exponentially toward personalized model.
Data Overfitting	Cross-validate against recent task splits; regularize small user datasets toward global priors.
Manual Overrides	Log overrides as explicit feedback features; display model confidence intervals.
Data Privacy	Exclude raw task payload content; store only anonymized timestamps, categories, and metrics.

10 Implementation Timeline

Timeline	Milestone Focus
Weeks 1 to 2	Requirements definition, schema specification, authentication and API core setup.
Weeks 3 to 4	Baseline bias regression implementation and task CRUD operational workflows.
Weeks 5 to 7	Circadian productivity engine, context-switch metrics, and inference pipeline.
Weeks 8 to 9	Deadline proximity integration, auto-rebuilder development, and notification engine.
Weeks 10 to 11	Analytics dashboard implementation, performance tuning, and KPI benchmarking.
Week 12	Comprehensive integration testing, system hardening, and project defense.

11 Summary

TimeLens AI converts task management into an adaptive, data-driven forecasting ecosystem. By quantifying individual operational bias, circadian performance curves, context-switch overhead, and deadline behavior, the platform replaces manual scheduling with dynamic, statistically backed temporal planning evaluated via clear, empirical KPIs.